作业 3

大模型分析数据（电信行业用戶流失分析）

任务介绍:

结合机器学习算法和大模型分析思维，深入探索和预测电信行业用戶流失的主要驱动因素。通过对原始数据集（下图展示部分数据）进行数据预处理、特征工程、模型选择、评估以及结果可视化等操作，最后对电信行业的用戶流失因素进行总结分析。

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
1		state	account_le	area_code	internationa	voice_mail_	number_vm	total_day_n	total_day_c	total_day_c	total_eve_n	total_eve_c	total_eve_c	total_night_	total_night_	total_night_	total_intl_m	total_intl_ca	total_intl_ch	number_cu	churn
2	1	KS	128	area_code_	no	yes	25	265.1	110	45.07	197.4	99	16.78	244.7	91	11.01	10	3	2.7	1	no
3	2	OH	107	area_code_	no	yes	26	161.6	123	27.47	195.5	103	16.62	254.4	103	11.45	13.7	3	3.7	1	no
4	3	NJ	137	area_code_	no	no	0	243.4	114	41.38	121.2	110	10.3	162.6	104	7.32	12.2	5	3.29	0	no
5	4	OH	84	area_code_	yes	no	0	299.4	71	50.9	61.9	88	5.26	196.9	89	8.86	6.6	7	1.78	2	no
6	5	OK	75	area_code_	yes	no	0	166.7	113	28.34	148.3	122	12.61	186.9	121	8.41	10.1	3	2.73	3	no
7	6	AL	118	area_code_	yes	no	0	223.4	98	37.98	220.6	101	18.75	203.9	118	9.18	6.3	6	1.7	0	no
8	7	MA	121	area_code_	no	yes	24	218.2	88	37.09	348.5	108	29.62	212.6	118	9.57	7.5	7	2.03	3	no
9	8	MO	147	area_code_	yes	no	0	157	79	26.69	103.1	94	8.76	211.8	96	9.53	7.1	6	1.92	0	no
10	9	LA	117	area_code_	no	no	0	184.5	97	31.37	351.6	80	29.89	215.8	90	9.71	8.7	4	2.35	1	no
11	10	WV	141	area_code_	yes	yes	37	258.6	84	43.96	222	111	18.87	326.4	97	14.69	11.2	5	3.02	0	no
12	11	IN	65	area_code_	no	no	0	129.1	137	21.95	228.5	83	19.42	208.8	111	9.4	12.7	6	3.43	4	yes
13	12	RI	74	area_code_	no	no	0	187.7	127	31.91	163.4	148	13.89	196	94	8.82	9.1	5	2.46	0	no
14	13	IA	168	area_code_	no	no	0	128.8	96	21.9	104.9	71	8.92	141.1	128	6.35	11.2	2	3.02	1	no
15	14	MT	95	area_code_	no	no	0	156.6	88	26.62	247.6	75	21.05	192.3	115	8.65	12.3	5	3.32	3	no
16	15	IA	62	area_code_	no	no	0	120.7	70	20.52	307.2	76	26.11	203	99	9.14	13.1	6	3.54	4	no
17	16	NY	161	area_code_	no	no	0	332.9	67	56.59	317.8	97	27.01	160.6	128	7.23	5.4	9	1.46	4	yes

具体操作如下

一、数据预处理与清洗

核心目标：让数据变得干净、可用，并初步理解数据结构。

1. 数据集简介

你

请罗列数据中的所有字段，并解释。

格式如： Hours_Studied（学习小时数）：每周用于学习的小时数。

Gemini 3 Pro Thinking 深度思考完成

基于您提供的 `customer_churn.csv` 数据集内容，以下是所有字段的罗列及其解释：

- **[Unnamed: 0] / Index（索引/用户ID）**：数据表中的行号或客户的唯一标识符。
- **state（州名）**：用户居住的美国州名缩写（例如：KS 代表堪萨斯州，OH 代表俄亥俄州）。
- **account_length（账户时长）**：客户持有该账户的天数（表示客户入网的时间长度）。
- **area_code（区号）**：客户电话号码所在的区域代码（例如：area_code_415）。
- **international_plan（国际套餐）**：客户是否开通了国际长途套餐（yes 表示已开通，no 表示未开通）。
- **voice_mail_plan（语音信箱套餐）**：客户是否开通了语音信箱功能（yes 表示已开通，no 表示未开通）。
- **number_vmail_messages（语音信箱数量）**：客户语音信箱中存储的留言数量。
- **total_day_minutes（白天通话时长）**：客户在白天时段的通话总分钟数。
- **total_day_calls（白天通话次数）**：客户在白天时段拨打的电话总次数。
- **total_day_charge（白天通话费用）**：客户在白天时段产生的通话总费用。
- **total_eve_minutes（傍晚通话时长）**：客户在傍晚时段的通话总分钟数。
- **total_eve_calls（傍晚通话次数）**：客户在傍晚时段拨打的电话总次数。
- **total_eve_charge（傍晚通话费用）**：客户在傍晚时段产生的通话总费用。
- **total_night_minutes（夜间通话时长）**：客户在夜间时段的通话总分钟数。
- **total_night_calls（夜间通话次数）**：客户在夜间时段拨打的电话总次数。
- **total_night_charge（夜间通话费用）**：客户在夜间时段产生的通话总费用。
- **total_intl_minutes（国际通话时长）**：客户进行国际长途通话的总分钟数。
- **total_intl_calls（国际通话次数）**：客户进行国际长途通话的总次数。
- **total_intl_charge（国际通话费用）**：客户进行国际长途通话产生的总费用。
- **number_customer_service_calls（客服电话次数）**：客户拨打运营商客户服务热线的次数（这是预测流失的一个关键指标）。
- **churn（流失状态）**：目标变量，表示客户是否已经流失（yes 表示已流失，no 表示仍为客户）。

数据集中的变量包括以上内容，该内容显示该数据集包含了关于电信行业用户流失的多维度信息，涉及影响电话套餐、通话时长、通话费用等多种因素。

2. 理解数据集

你

现在要做的是 理解数据集：分析数据集中的 3333 条记录和 21 个特征，理解各个特征的意义并进行合理的分类。重点分析目标变量 **churn** （用户流失）与其他变量之间的关系。

这一步是为了从数据中“讲故事”，找出“为什么用户会流失”，而不是仅仅列出数字。大模型部分输出结果如下图，这为接下来的数据处理提供了大致的思路。

第一步：分析数据处理意图

分析：
您希望对数据集进行“探索性数据分析（EDA）”。

1. **理解与分类：** 将杂乱的 21 个特征梳理成有业务逻辑的类别。
2. **关系探索：** 重点挖掘 `churn` （流失）与其他特征（如通话时长、套餐类型、客服电话）之间的关联。
3. **分布可视化：** 使用统计图形展示数据的形状、异常值和集中趋势。

第二步：特征分类与理解

为了更好地分析，我将这 21 个特征分为以下 **4 个逻辑类别**：

1. 目标变量 (Target Variable)

- `churn` : **[核心]** 用户是否流失。这是我们要分析和预测的对象。

2. 用户画像与服务信息 (Customer Profile & Services)

- `state` : 地理位置。
- `area_code` : 电话区号。
- `account_length` : 用户入网时长（老用户还是新用户）。
- `international_plan` : 是否有国际套餐（Yes/No）。
- `voice_mail_plan` : 是否有语音信箱套餐（Yes/No）。
- `number_vmail_messages` : 语音留言数量。

3. 用户行为数据 - 通话用量 (Usage Metrics)

- *这部分数据存在高度的结构化重复（分钟数、次数、费用），分为四个时段：*
  - **白天 (Day):** `total_day_minutes` , `total_day_calls` , `total_day_charge`
  - **傍晚 (Eve):** `total_eve_minutes` , `total_eve_calls` , `total_eve_charge`
  - **夜间 (Night):** `total_night_minutes` , `total_night_calls` , `total_night_charge`
  - **国际 (Intl):** `total_intl_minutes` , `total_intl_calls` , `total_intl_charge`
- *注意：* 通常“分钟数”和“费用”是完全正相关的（多重共线性），分析时主要看其中一个即可。

4. 客户交互 (Customer Interaction)

- `number_customer_service_calls` : **[关键]** 拨打客服电话的次数。这通常反映了用户的不满程度。

3. 数据清洗

数据清洗是为了删除无关变量（如 Unnamed:0 ），查找缺失值和异常值并进行处理，并将 yes/no 等字符串标签转换为数值（0/1）方便后续训练模型。

你

现在我要进行数据清洗，请帮我写一段在JupyterNotebook中实现的代码来完成以下步骤：
读取 'customer_churn.csv' 文件，删除无关变量（如 Unnamed:0 ），处理缺失数据和异常值，进一步将 'churn', 'international_plan', 'voice_mail_plan' 这几列的 'yes'/'no' 转换为 1/0。最后打印前 5 行数据和数据的基本信息 (info)。

Gemini 3 Pro Thinking

```py
# 导入必要的库
import pandas as pd
import numpy as np

# 1. 读取数据
# 假设 customer_churn.csv 与当前 notebook 在同一目录下
df = pd.read_csv('customer_churn.csv')

# 2. 删除无关变量
# 作业中特别提到 Unnamed:0 是无关变量
# errors='ignore' 可以防止如果列不存在时报错
df = df.drop(columns=['Unnamed: 0'], errors='ignore')

# 3. 处理缺失数据
# 检查缺失值
print("处理前缺失值情况: \n", df.isnull().sum())

# 简单的处理策略：如果缺失值很少，直接删除；或者用均值/众数填充
# 这里我们采用删除含有缺失值的行的策略（视具体数据量而定，也可选择 fillna）
```

通过交互，Gemini 给出了实现代码（此处仅展示部分代码，具体完整代码见 .ipynb 文件），运行结果如下，结果显示该数据集数据完整，无缺失值：

```
处理前缺失值情况:
 state                           0
account_length                  0
area_code                       0
international_plan              0
voice_mail_plan                 0
number_vmail_messages           0
total_day_minutes               0
total_day_calls                 0
total_day_charge                0
total_eve_minutes               0
total_eve_calls                 0
total_eve_charge                0
total_night_minutes             0
total_night_calls               0
total_night_charge              0
total_intl_minutes              0
total_intl_calls                0
total_intl_charge               0
number_customer_service_calls   0
churn                           0
dtype: int64
```

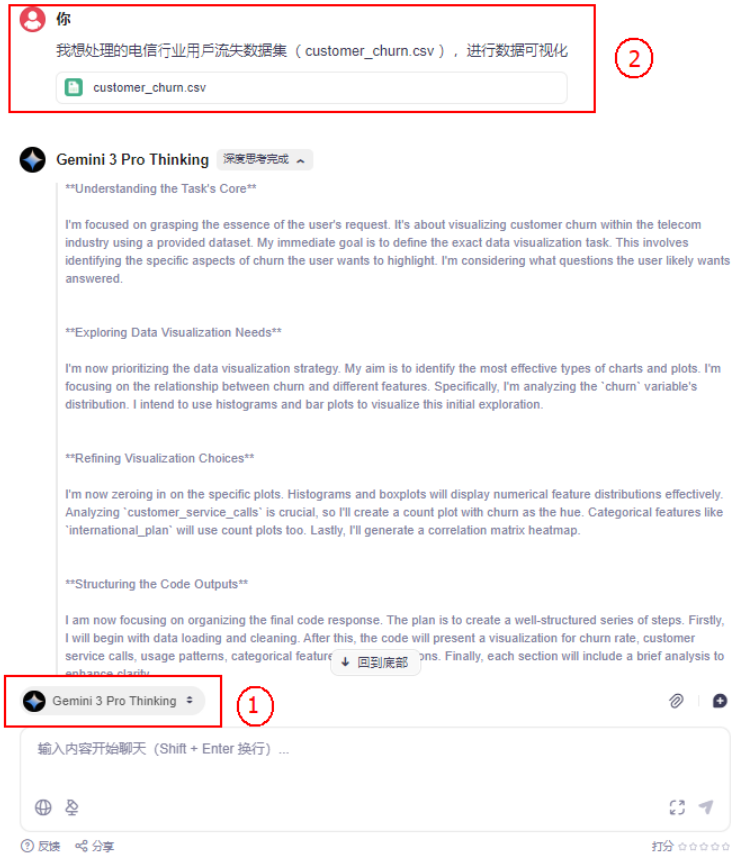
```
前 5 行数据预览:
```

	state	account_length	area_code	international_plan	voice_mail_plan	number_vmail_messages	total_day_minutes	total_day_calls	total_day_charge	total
0	KS	128	area_code_415	0	1	25	265.1	110	45.07	
1	OH	107	area_code_415	0	1	26	161.6	123	27.47	
2	NJ	137	area_code_415	0	0	0	243.4	114	41.38	
3	OH	84	area_code_408	1	0	0	299.4	71	50.90	
4	OK	75	area_code_415	1	0	0	166.7	113	28.34	

```
==========
数据基本信息 (Info):
==========
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3333 entries, 0 to 3332
Data columns (total 20 columns):
 #   Column                        Non-Null Count  Dtype
---  ------                        --------------  -----
 0   state                         3333 non-null   object
 1   account_length                3333 non-null   int64
 2   area_code                     3333 non-null   object
 3   international_plan            3333 non-null   int64
 4   voice_mail_plan               3333 non-null   int64
 5   number_vmail_messages         3333 non-null   int64
 6   total_day_minutes             3333 non-null   float64
 7   total_day_calls               3333 non-null   int64
 8   total_day_charge              3333 non-null   float64
 9   total_eve_minutes             3333 non-null   float64
 10  total_eve_calls               3333 non-null   int64
 11  total_eve_charge              3333 non-null   float64
 12  total_night_minutes           3333 non-null   float64
 13  total_night_calls             3333 non-null   int64
 14  total_night_charge            3333 non-null   float64
 15  total_intl_minutes            3333 non-null   float64
 16  total_intl_calls              3333 non-null   int64
 17  total_intl_charge             3333 non-null   float64
 18  number_customer_service_calls 3333 non-null   int64
 19  churn                         3333 non-null   int64
dtypes: float64(8), int64(10), object(2)
memory usage: 520.9+ KB
None

检查转换后的唯一值:
Churn: [0 1]
International Plan: [0 1]
Voice Mail Plan: [1 0]
```

二、探索性数据分析（EDA）与可视化

核心目标：完成图表绘制，并发现流失规律。



为了能够更精准的通过输入得到我们想要的结果和图片，在进行数据可视化前，先通过输入提示词更新 Gemini 的记忆，定义好每次输出结果的格式。

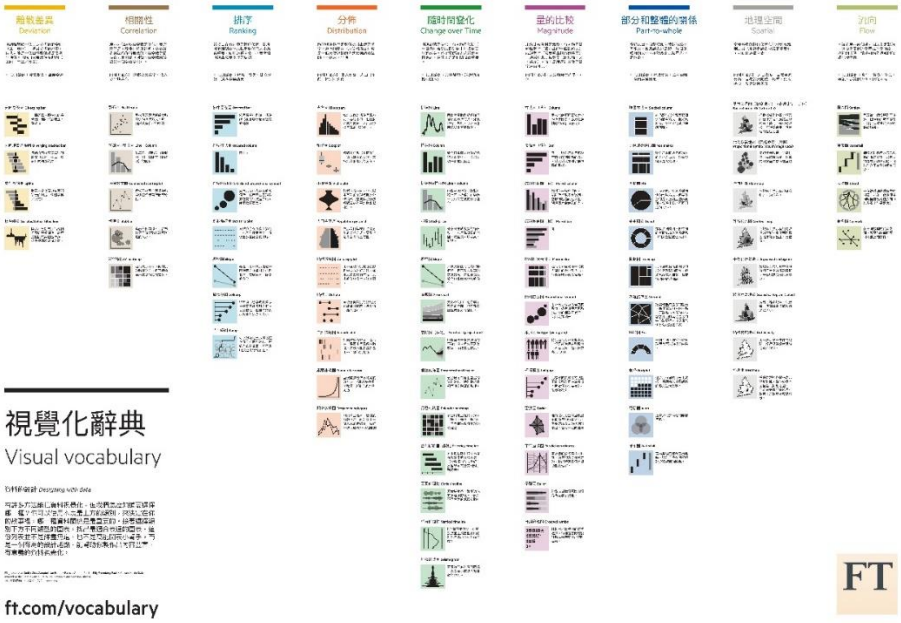
Gemini 3 Pro Thinking

Visual-vocabulary-cn-traditional.pdf
1.39 MB

记住这些可视化的每一个点，每次我告诉你我想处理什么数据的时候，你都要按照以下步骤来一步一步执行：
先分析我处理这个数据是干什么用的，然后问我是不是跟你想的一样，如果不是就按照我的想法来。
思考处理这个数据最好用上面的哪种可视化来处理，并告诉我为什么可以这样处理，询问我要不要换一种别的可视化方法并且提供给我别的方案，也要告诉我为什么可以用别的方案。
处理数据时不仅要有数据表格，还要有对应的图片，例如直方图等，这个图表要根据上面我所提供的九大类，以及你前面步骤的思考来完成绘制。
最后再给我一个结果分析。

反馈 分享 打分

（该辞典为本次画图参考的重点！）



此处仅展示部分内容：

收到，我已经学习了您提供的《Visual Vocabulary（视觉化辞书）》指南。针对您之前上传的 `customer_churn.csv` （电信用户流失数据集），我将严格按照您要求的步骤进行处理。

以下是基于该数据集和视觉化辞书的执行流程：

第一步：分析数据处理意图

分析：
这个数据集包含了用户的基础信息（州、地区）、行为数据（通话时长、费用、短信量）以及最终的分类标签（Churn: yes/no）。
我认为您处理这个数据的核心目的是 **"诊断与发现"**。您希望找出是哪些行为特征或属性导致了用户流失。
即：流失用户和留存用户在行为上有什么本质区别？

确认：
我的分析是否与您的想法一致？（假设您点头确认，我们继续下一步。如果您原本想做的是比如"预测各州的总收入"等其他目的，请随时纠正我。）

第二步：选择最佳可视化方案（Primary Option）

基于《Visual Vocabulary》中的分类，我认为处理这个数据以达到"寻找流失原因"的最佳可视化方式是：

推荐方案：Distribution（分布）类别下的 Boxplot（箱形图）

理由：

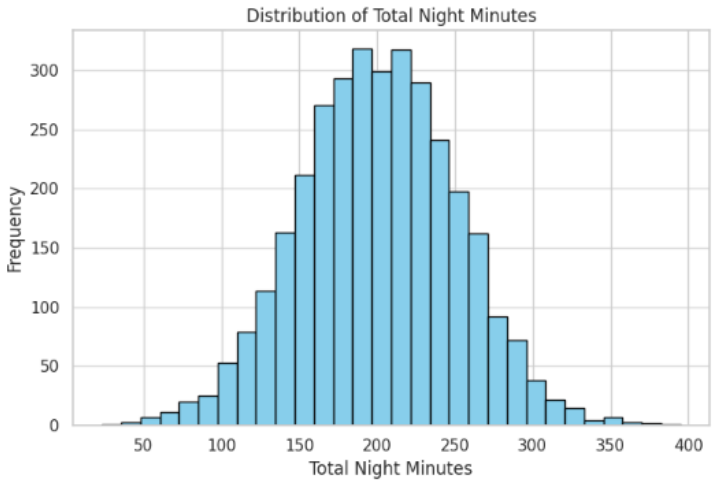
1. **依据指南：** 指南中提到，当需要"概括多个数列的分付"或比较"不同类别数据的变化"时，箱形图非常有效。
2. **适用性：** 我们的核心问题是比较两类人（Churn=Yes 和 Churn=No）。我们需要看这两类人在数值型变量（如 `total_day_minutes` 白天通话 ↓ 回到底部 `day_charge` 费用）上的分布是否有显著差异。

1. 可视化数据分布并揭示变量之间的关系

① 直方图可以查看变量的频率分布，了解数据是否符合正态分布，并展示分布的集中趋势、偏差和范围。

你

请用直方图展示变量 total_night_minutes 的分布趋势，给出具体分析结果。



具体分析如下：

1. 统计形态分析

- **正态分布（Normal Distribution）：**
  - 这是最显著的特征。整个分布呈现出极其标准的"**"钟形曲线"（Bell Curve）**。
  - 这意味着夜间通话时长是一个非常自然、随机的变量，受到多种独立因素影响，最终在宏观上呈现出稳定的正态分布。
- **集中趋势（Central Tendency）：**
  - **均值/中位数：**峰值大约位于 **200 分钟**左右。说明绝大多数用户的夜间通话时长集中在 200 分钟上下。
- **离散程度（Spread）：**
  - 数据主要分布在 **100 到 300 分钟**之间。
  - **极值：**低于 50 分钟（极低频使用）和高于 350 分钟（极高频使用）的用户都非常少，属于长尾部分的异常值（Outliers）。
- **对称性：**
  - 图形非常对称，没有明显的左偏或右偏（Skewness），说明用户群体在夜间使用习惯上非常均衡。

2. 与业务场景的关联（Telecom Churn）

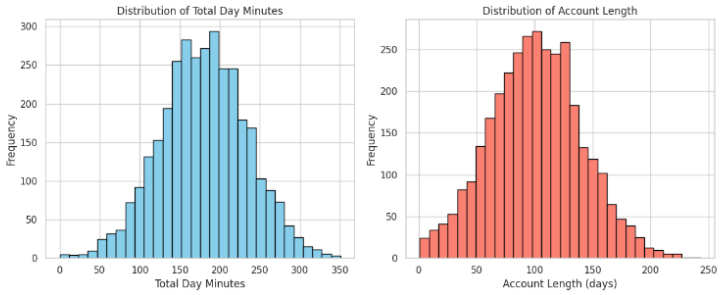
结合之前的模型特征重要性分析，我们可以得出以下洞察：

- **特征稳定性：**
  - 由于数据分布如此完美地符合正态分布，说明**夜间通话行为是非常稳定的。**这种稳定性通常意味着它不像"日间通话"那样具有极端的两极分化。
- **对流失的影响（次要因素）：**
  - 回顾之前的特征重要性图表，`total_night_minutes` 排在第 8 位左右，重要性远低于 `total_day_minutes`。
  - **原因推测：**
    1. **资费差异：**电信运营商通常在夜间时段提供极其便宜的费率，甚至"夜间免费"。因此，即使是右侧长尾（>300分钟）的用户，其账单金额也不会像白天打得多那样暴涨。
    2. **用户心理：**用户在夜间通话通常是为了私人社交（与家人朋友煲电话粥），这种情感连接反而可能增加对号码的依赖度，不像白天商务通话那样纯粹是成本支出。
- **异常检测价值：**
  - 虽然整体影响不如白天大，但在右侧极端尾部（>350分钟）的用户仍需关注。如果这部分用户没有办理"夜间包月"，他们依然可能面临高额账单风险。

其余操作类似。

你

请用直方图展示 total_day_minutes（通话时长）和 account_length（账户时长）的分布，给出具体分析结果。



左图：Distribution of Total Day Minutes（白天通话时长分布）

- **集中趋势（Central Tendency）：**
  - 数据的峰值（最高点）大约在 **175 到 180 分钟**之间。这说明绝大多数用户的白天通话时长都在 3 小时左右。
  - 分布非常对称，均值（Mean）和中位数（Median）应该非常接近。
- **离散程度（Spread）：**
  - 数据范围大致从 0 分钟到 350 分钟。
  - 大部分数据集中在 **100 到 250 分钟**这个区间内。
- **业务洞察：**
  - **极值关注：**虽然大部分用户是"普通用户"，但图表右侧（>300分钟）的长尾代表了**高频用户**。在流失分析中，这部分人非常关键，因为他们可能产生高额费用，如果对资费不满意，流失风险可能较高。
  - **低频用户：**左侧（<50分钟）的用户可能是不活跃用户，也需要关注是否因为不需要服务而流失。

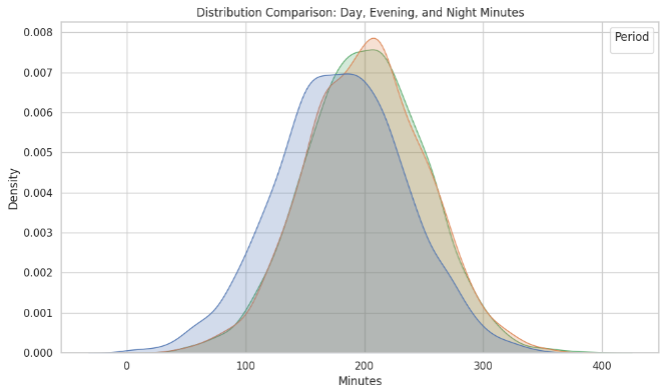
右图：Distribution of Account Length（账户持有时长分布）

- **集中趋势：**
  - 峰值大约在 **100 天**左右。说明大部分客户的入网时间在 3 个月多一点。
- **分布形态：**
  - 同样呈现正态分布，略微有一点点向右的尾巴，但整体很对称。
- **离散程度：**
  - 大部分用户的入网时间在 **50 到 150 天**之间。
  - 很少有新用户（接近 0 天）或极老的用户（超过 200 天）。
- **业务洞察：**
  - **数据集特性：**这个分布表明数据集是一个相对**成熟且稳定**的客户群。它不是一个充斥着大量新注册用户（如果是那样，图表会向左偏）的数据集。
  - **流失关联：**由于分布如此均匀，单纯的"入网时长"可能不是流失的决定性因素（即新老用户比例相当），但需要结合其他特征分析。例如，查看入网 100 天左右的用户是否到了"合约期满"的敏感期。

② 核密度图则比直方图更平滑，能够显示出数据分布的形态，适用于展示数据的总体趋势。

你

请在同一张密度图比较 total_day_minutes, total_eve_minutes, total_night_minutes 的分布差异，给出具体分析结果。



具体分析如下：

A. 正态分布特性 (Normality)

- **观察**：三条曲线都呈现出非常完美的**钟形（Bell Shape）**。
- **结论**：这表明白天、傍晚和夜间的通话时长都服从**正态分布（Normal Distribution）**。这对于后续的数据建模非常有理，许多模型（如高斯朴素贝叶斯、逻辑回归）在处理正态分布数据时表现更好。

B. 中心位置差异 (Shift in Central Tendency)

- **白天（蓝色区域）**：峰值（众数/均值）大约在 **180分钟** 左右。
- **傍晚/夜间（橙/绿区域）**：峰值大约在 **200-210分钟** 左右。
- **结论：用户在白天打电话的平均时长要短于傍晚和夜间。** 这是一个显著的差异。

C. 分布的重叠 (Overlap)

- **傍晚与夜间的高度重合**：橙色和绿色曲线的形态、峰值位置几乎完全一致，重叠度极高。这说明用户在傍晚和夜间的通话行为模式是非常相似的。
- **白天与其他的错位**：蓝色曲线明显向左偏移（数值更小）。

这张图清晰地揭示了：用户在傍晚和夜间的通话时长分布高度一致，且平均时长明显高于白天。在建模时，要注意白天分钟数虽然数值较小，但其单位价值（费用权重）可能更高。

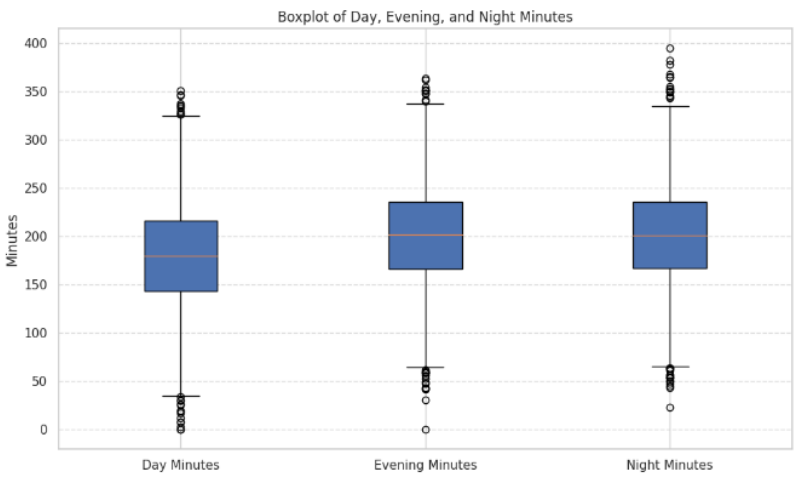
为什么会出现这种分布差异？这通常与当时的**资费策略**和**生活习惯**有关：

1. **资费影响行为**：在很多传统的电信套餐中，**白天（工作时间）的费率通常最高**，而傍晚和夜间往往有"闲时优惠"甚至免费时段。因此，用户倾向于在晚上煲电话粥，而在白天尽量长话短说，导致白天均值偏低（~180分钟 vs ~200分钟）。
2. **使用场景不同**：
   - **白天**：多为工作沟通或事务性通话，讲究效率，时间较短。
   - **晚/夜**：多为社交、家庭沟通，倾向于长时间的情感交流。
3. **流失分析的切入点**：
   - 既然白天费率高且大家普遍打得少，那么**那些在白天通话时长极高（位于蓝色曲线最右侧长尾，例如 >300分钟）的用户**，就是给公司贡献最多收入的**高价值客户**。
   - 如果这部分高价值用户的流失率高，说明公司的白天资费可能缺乏竞争力，或者服务质量在高峰期存在问题。

③ 箱形图是一种常用于显示数据分布和离散情况的图表。它通过展示数据的四分位数、最小值、最大值和异常值，帮助分析不同类别之间的差异和分布。

你

请用箱线图分析 total_day_minutes, total_eve_minutes, total_night_minutes 的离散程度，给出具体分析结果。



具体分析如下：

A. 中位数对比（橙色线位置）

- **Day Minutes（左）**：中位数大约在 **180** 附近。
- **Evening Minutes（中）** & **Night Minutes（右）**：中位数大约在 **200** 附近。
- **结论**：这再次印证了之前密度图的发现，即**用户在白天通话的平均时长明显少于傍晚和夜间。**

B. 离散程度与稳定性（箱体高度）

- 三个箱体的高度（IQR）看起来非常相似。
- **结论**：这意味着无论是在白天、傍晚还是夜间，用户通话习惯的**波动程度是相似的。**并没有出现"白天大家都很统一，晚上大家差异很大"的情况。

C. 异常值分析（Outliers - 关键点）

- **上下两端都有异常值**：在三组数据中，箱须的上方和下方都存在大量的圆圈。
- **极高值（High Outliers）**：
  - 注意看 **Night Minutes** 的最高点，接近 **400分钟**，比 Day 和 Evening 的最高点都要高。
  - 这意味着存在一部分**"夜猫子"用户**或特定群体，他们在夜间有极高频的通话需求。
- **极低值（Low Outliers）**：
  - 接近 0 分钟的用户在三个时段都存在。这部分人可能是仅仅保留号码但不怎么使用的"休眠用户"。

这张图表明数据的分布是健康的，三个时段的波动性一致。但白天通话时长的中位水平较低。最需要关注的是图表上方那些代表高频使用的异常点，他们是潜在的高价值风险客户。

箱线图最强大的功能就是**识别异常值**，这对于流失预测非常有价值：

1. **针对"上方异常点"的高价值维护：**
   - 图表中那些位于箱须上方的圆圈（例如白天通话 > 325分钟，或夜间 > 350分钟的用户），是**极端高频用户。**
   - 在电信业务中，如果按分钟计费，这些人是**Top Revenue（高收入）**贡献者。
   - **策略**：需要检查这部分人的流失率。如果他们流失，对营收的打击是巨大的。需要确认他们是否经常因为费用过高而投诉。
2. **夜间通话的特殊性：**
   - 夜间出现了接近 400 分钟的极端值。这可能暗示某些特定类型的用户（如跨时区拨打国际长途、或者年轻人群体）。如果这部分人流失率高，可能需要推出"夜间无限畅聊包"之类的产品。
3. **数据清洗的提示：**
   - 虽然这些点是"异常值"，但在电信数据中它们通常是**真实的**（有人确实打这么多电话），而不是数据错误。因此在建模时，**不要轻易删除这些异常值**，因为它们往往蕴含着最重要的业务特征。

其余操作类似。

你

请用密度图分别分析 total_day_minutes 和 churn 、 total_eve_minutes 和 churn 、 total_night_minutes 和 churn 之间的关系，给出具体分析结果。

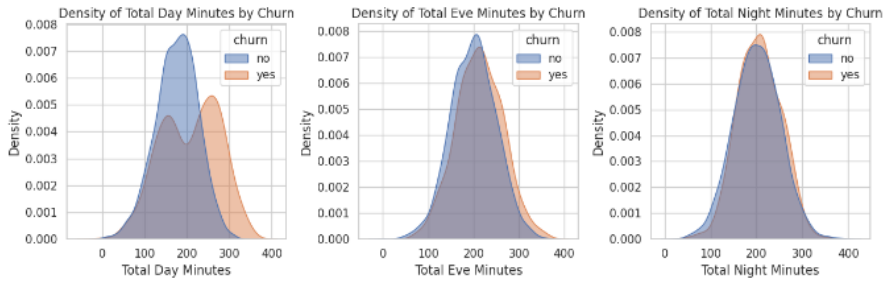


图1（左）：Total Day Minutes by Churn（白天通话时长）—— ⭐ 最关键发现

- **形态差异：**
  - **留存用户（蓝）**：呈现标准的正态分布，峰值集中在 **175分钟** 左右。
  - **流失用户（橙）**：分布形态非常不同，显得比较宽，且呈现出**双峰（Bimodal）** 或者明显的**右偏**特征。它的主要重心明显偏向右侧，峰值在 **260-270分钟** 附近。
- **分界线**：注意看 **225分钟** 右侧，橙色面积远大于蓝色面积。这意味着，**一旦用户的白天通话时长超过 225 分钟，他流失的概率就大幅增加。**
- **结论**：白天通话时长是预测流失的**强特征**。高频白天通话与流失有极强的相关性。

图2（中）：Total Eve Minutes by Churn（傍晚通话时长）

- **形态差异：**
  - 蓝色和橙色曲线的形状非常相似，都是正态分布。
  - 橙色曲线（流失）略微向右偏移，峰值大约在 210 分钟，而蓝色在 200 分钟。
- **重叠度**：重叠面积非常大。虽然流失用户在傍晚打得稍微多一点点，但区分度不高。
- **结论**：傍晚通话时长是一个**弱特征**，对流失有轻微影响，但不是决定性的。

图3（右）：Total Night Minutes by Churn（夜间通话时长）

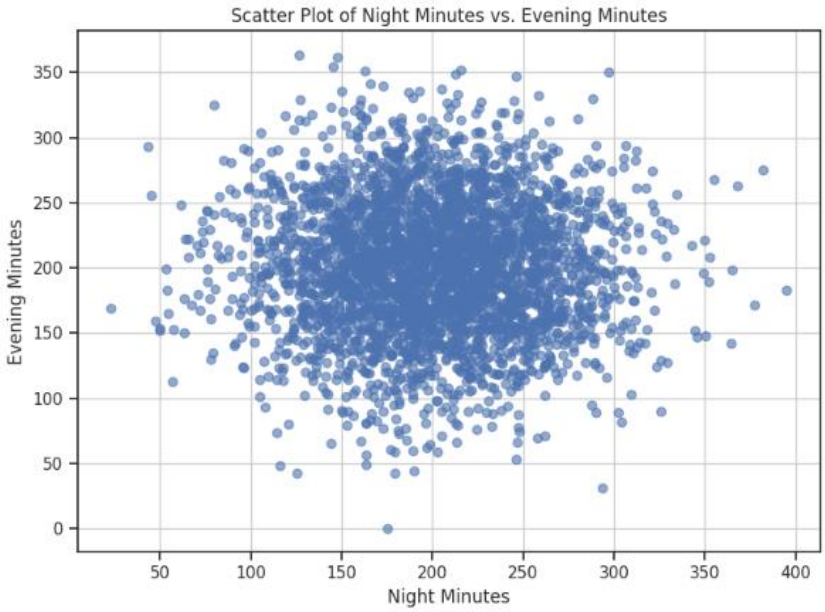
- **形态差异：**
  - 蓝色和橙色曲线几乎**完全重合**。
- **结论：夜间通话时长与流失几乎无关。**无论用户晚上打多久电话，都不能说明他是否会流失。在构建模型时，这个特征的权重可能会很低。

通过这组图可以发现，流失主要由白天的高频通话驱动，而与夜间通话行为无关。这很可能是由于白天的高昂资费导致的。

④ 散点图可以帮助我们直观地查看两个变量之间的关系。在散点图的基础上加入回归线就可以更好的展示两个变量间的趋势。

你

请用散点图分析 total_night_minutes 和 total_eve_minutes 之间的关系，给出具体分析结果。



结果显示用户的晚间习惯和深夜习惯是两码事，互不干涉。虽然这两个特征在技术上很干净（无共线性），但由于缺乏明显的异常聚类，它们可能只能作为辅助特征，而非核心的流失判定依据。

具体分析如下：

1. 核心统计发现：零相关性 (Zero Correlation)

- **形态特征**：图中的数据点呈现出一个标准的**圆形/椭圆形团块（Blob）**，没有任何明显的倾斜方向（既没有向右上倾斜，也没有向右下倾斜）。
- **统计含义**：这意味着"晚间通话时长"与"夜间通话时长"之间**几乎不存在线性相关性**（Pearson 相关系数 $r \approx 0$）。
- **解读**：
  - 一个用户在晚间（Evening）打多少电话，完全无法预测他在夜间（Night）会打多少电话。
  - 这两个变量在统计上是**相互独立**的。

2. 数据分布特征：双重正态分布

- **中心密集**：数据点最密集的区域集中在图表中心（Night ≈ 200, Evening ≈ 200）。
- **边缘稀疏**：越往四周扩散，点越稀疏。
- **验证结论**：结合之前的直方图，这进一步证实了**Night Minutes** 和 **Evening Minutes** 各自都服从**正态分布（高斯分布）**。当两个独立的、服从正态分布的变量放在一起作图时，就会形成这种典型的"靶心"状分布。

3. 业务洞察 (Business Insights)

- **用户行为的独立性**：
  - **晚间（Evening）**通常是下班后的社交活跃期（约饭、联系家人）。
  - **夜间（Night）**通常是深夜时段。
  - 这张图说明：**"话痨"属性是不跨时段继承的。**那种晚间话务量大的用户，并不一定会在深夜继续煲电话粥；反之，深夜喜欢打电话的人，晚间也不一定活跃。这代表了两种截然不同的生活习惯或用户画像。
- **无明显聚类 (No Clustering)**：
  - 图中没有出现分离的"小岛"或异常群体。这意味着我们很难简单地通过这两个维度的组合来划分出特殊的细分客群（比如没有明显的"晚间沉默但夜间爆发"的怪异群体）。

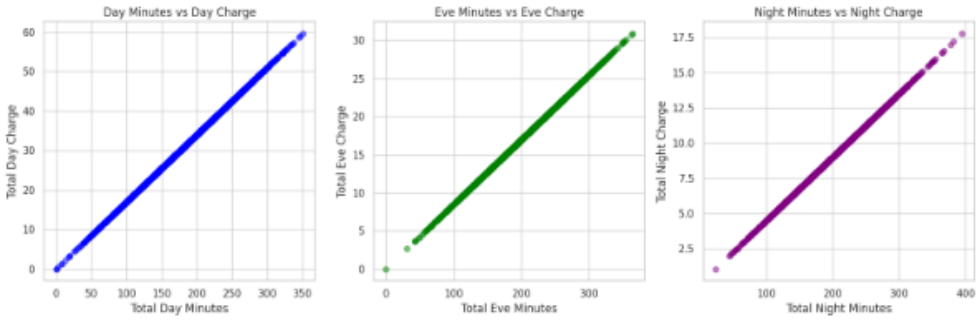
4. 对建模的影响

- **无多重共线性 (No Multicollinearity)**：
  - 这是个好消息。因为这两个特征互不相关，它们可以同时放入逻辑回归或线性模型中，而不会引起多重共线性问题。
  - 它们为模型提供了**正交（Orthogonal）**的信息——即每个特征都在从不同的角度描述用户。
- **预测价值有限**：
  - 虽然它们互不干扰，但鉴于形状如此规则且集中，说明大部分用户都表现得很"中庸"。除非结合其他特征（如流失标签），否则单看这张图，很难找到能区分"流失用户"与"留存用户"的边界。这解释了为什么在之前的特征重要性排行中，Evening 和 Night 的权重都不如 Day Minutes 高。

其余操作类似。

你

请用散点图分别分析 total_day_minutes 和 total_day_charge 、 total_eve_minutes 和 total_eve_charge 、 total_night_minutes 和 total_night_charge 之间的关系，给出具体分析结果。



- **形态**：三张图中的点都排列成一条**完美的直线**，且都经过原点 (0,0)。
- **统计含义**：这意味着"分钟数"和"费用"之间的相关系数（Pearson Correlation Coefficient）为 **1.0**（或者非常接近 1.0）。
- **业务逻辑**：这证实了该电信公司的计费方式是**固定单价计费**。
  - 公式：费用 = 通话时长 × 每分钟单价
  - 没有复杂的阶梯定价、套餐包月额度或首分钟免费等情况，否则线条会出现弯曲或断层。

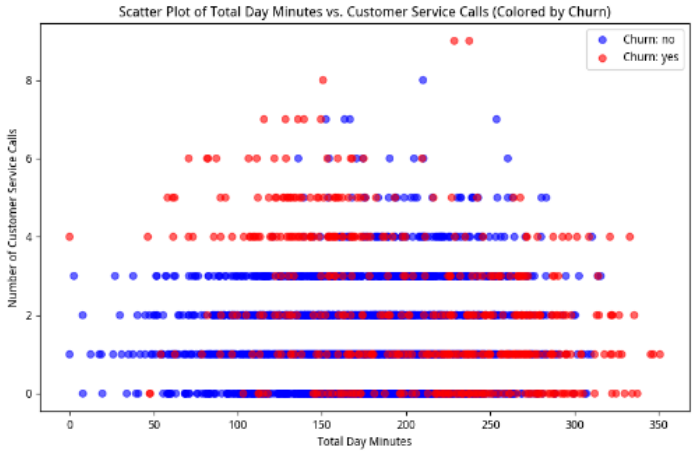
这组图在数据预处理和特征工程阶段具有**决定性意义**：

- **多重共线性（Multicollinearity）警报**：
  由于分钟数和费用是完全线性相关的，它们包含的**信息量是完全重复的**。
- **对模型的影响**：
  - 对于**逻辑回归（Logistic Regression）**等线性模型，这会导致"矩阵奇异"或计算不稳定，严重影响模型性能。
  - 对于**决策树/随机森林**，虽然模型能跑通，但这会分散特征重要性（Feature Importance），比如模型可能会把 50% 的重要性给"分钟"，50% 给"费用"，导致解释变难。

分钟数是用户的原始行为数据，更直观；费用只是分钟数的数学衍生品，在将数据输入机器学习模型之前应删除，删掉它不会丢失任何预测信息。

你

请用散点图分析 total_day_minutes 和 number_customer_service_calls 之间的关系，并根据 'churn' 列对点进行着色，给出具体分析结果。



该结果清晰地展示了流失（红色点）并非随机分布，而是集中在两个特定的"高危区域"。这意味着流失是由两个完全不同的原因驱动的，发现了两类截然不同的流失人群，并确立了"安全区"：

A. 第一类人群：不满意的投诉者 (The "Unhappy" Churners)

- **位置**：图表的**上半部分**（Y轴较高）。
- **特征**：**客服电话次数 > 3次**。
- **观察**：
  - 当客服电话次数为 0, 1, 2, 3 时，绝大多数点是蓝色的（留存）。
  - **一旦客服电话达到 4 次（Y=4）**，红点（流失）的比例开始急剧上升。
  - **当客服电话达到 5 次及以上时**，几乎全是一片红色。
- **业务解读**：这部分用户流失是因为**服务问题**或**技术故障**。他们打电话是为了解决问题，但显然问题没解决或体验极差，导致他们最终失去耐心而离开。

B. 第二类人群：高消费的沉默者 (The "Price-Sensitive" Churners)

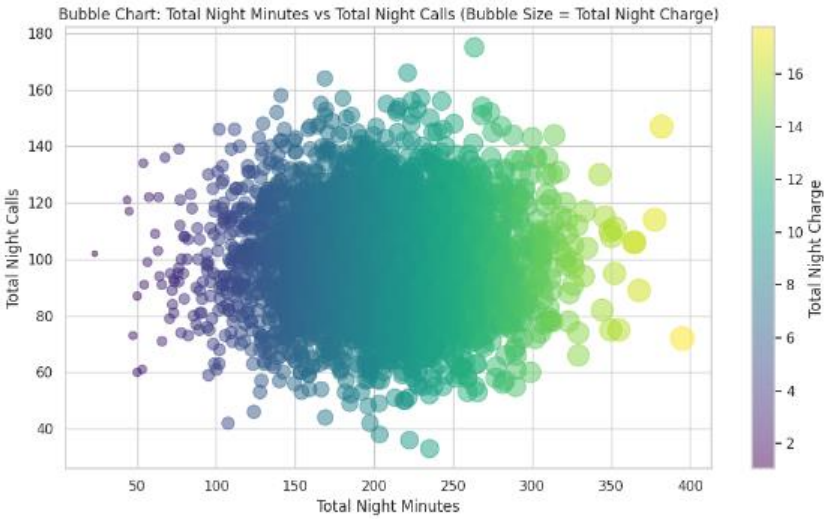
- **位置**：图表的**右下角**（X轴靠右，Y轴较低）。
- **特征**：**白天通话时长 > 260分钟**，但**客服电话次数 < 4次**。
- **观察**：
  - 请注意看 X 轴 260 分钟以右的区域。即使这些用户的客服电话很少（在 0-3 次之间），依然出现了密集的红点。
- **业务解读**：这部分用户流失是因为**价格问题**。他们打了很多电话，导致账单金额很高（白天费率贵）。他们没有因为服务质量投诉（电话打得少），而是因为找到更便宜的竞争对手而悄悄离开了。

- **位置**：图表的**左下角**矩形区域。
- **范围**：**白天通话时长 < 260分钟 且 客服电话次数 < 4次**。
- **观察**：在这个区域内，绝大多数点都是蓝色的。
- **结论**：这是公司的核心稳固客户群。只要用户的使用量适中，且没有遇到太多麻烦需要找客服，他们的留存率是非常高的。

⑤ 气泡图是散点图的扩展，它不仅展示两个变量之间的关系，还可以通过气泡的大小来表示第三变量。三维散点图可以同时展示三个变量之间的关系，更直观地理解三个变量的相互作用。

你

请用气泡图展示 total_night_minutes 与 total_night_calls 之间的关系，并用气泡大小表示 total_night_charge，给出具体分析结果。



具体分析如下：

以下是对该图表的详细分析结论，揭示了变量之间的数学关系和业务逻辑：

1. 关键发现：完全的线性相关性 (Minutes vs. Charge)

- **视觉证据**：请观察颜色的变化。颜色**严格地从左向右**过渡（从紫色的低话费 → 蓝绿色 → 黄色的高话费）。同时，气泡的大小也是从左向右逐渐变大。
- **数学含义**：这说明 **"通话时长" (Total Night Minutes)** 与 **"话费" (Total Night Charge)** 之间存在**完美的正相关关系** (Correlation ≈ 1.0)。
- **业务逻辑**：这符合电信行业的计费规则——**按分钟计费**。
  - 公式推测：$Charge = Minutes \times Rate$。
  - 因为颜色带是垂直切分的（与Y轴无关），这证明话费仅取决于打了多久，跟打了多少个电话没关系。

2. 变量独立性：时长与次数无关 (Minutes vs. Calls)

- **视觉证据**：观察散点分布的形状。它是一个松散的**团块 (Blob)**，而不是一条对角线。
  - **右侧 (高时长)**：既有高通话次数的用户（右上），也有低通话次数的用户（右下）。
  - **顶部 (高次数)**：既有长话短说的用户（左上，紫色），也有煲电话粥的用户（右上，黄色）。
- **业务含义**：
  - **通话次数 (Calls)** 对于预测 **话费 (Charge)** 毫无帮助。
  - 这也意味着存在两种截然不同的高价值用户：
    - **"煲粥型"**（右上）：打得多次，每次都聊很久。
    - **"长聊型"**（右下）：只打一两个电话，但一聊就是几小时。

3. 数据科学与特征工程建议 (关键点)

这张图为后续的数据预处理提供了强有力的依据：

- **多重共线性警报 (Multicollinearity Alert)**：
  - 由于 $Minutes$ 和 $Charge$ 的相关性几乎为 1（颜色分布完全由 X 轴决定），**它们在数学上是冗余的。**
  - **操作建议**：在训练模型（特别是逻辑回归）之前，**必须删除其中一列**（通常删除 `Charge`，保留 `Minutes`，因为时长是用户行为，话费是计算结果）。如果不删除，会导致模型参数不稳定，降低解释性。

4. 业务风险提示

- **高危区域 (黄色气泡)**：
  - 图表右侧的黄色大球代表了夜间话费最高的用户。
  - 虽然夜间费率通常较低，但如果用户处于最右侧极端位置（>350分钟），依然会产生显著费用。
  - **策略**：这部分"黄色气泡"用户是夜间包月套餐的最佳推销对象。如果他们没有办理套餐，极易因为账单超出预期而流失。

这张图最核心的价值在于验证了特征间的共线性。它告诉我们："话费"只是"时长"的影子，在建模时应当移除"话费"字段，以避免数据冗余。

⑥ 接下来借助 Python 绘制散点图矩阵和相关性热力图。

散点图矩阵可以展示这四个变量两两之间的关系，帮助识别它们之间的潜在相关性；而相关性热力图通过计算相关系数矩阵，展示四个变量之间的相关性，并用热力图来表示它们之间的关系。

你

请用散点图矩阵和相关性热力图分析最关键的三个特征（白天通话时长、傍晚通话时长、客服电话次数）以及流失状态

以下仅展示部分代码，具体完整代码见 .ipynb 文件。

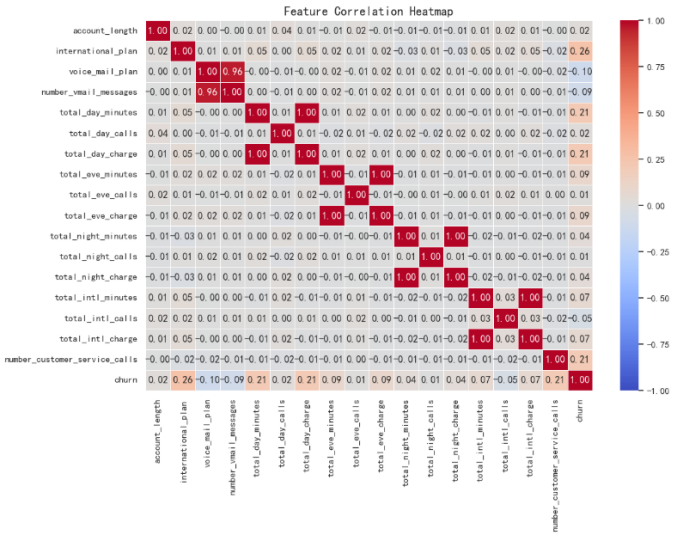
以下是代码实现和具体的分析逻辑。我选取了最关键的三**个特征**（白天通话时长、傍晚通话时长、客服电话次数）以及**流失状态**来进行深度观察。

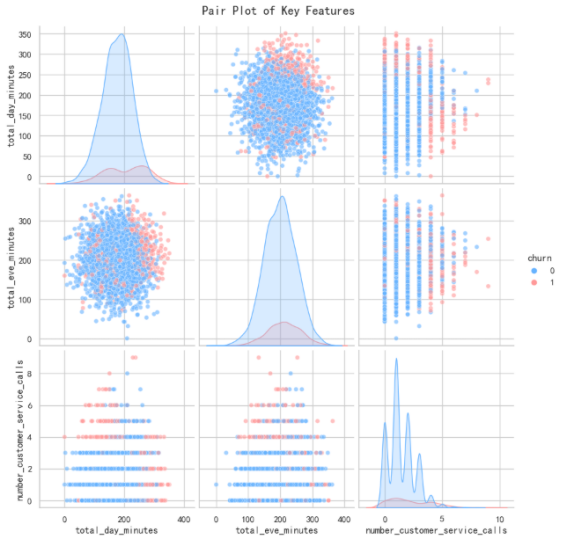
1. 代码实现

```py
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

# 假设 df 是你的数据框

# ------------------------------------------------------------
# 准备工作：将 Churn 转换为数字，以便在热力图中计算相关系数
# ------------------------------------------------------------
df_corr = df.copy()
# 假设 churn 列是 'yes'/'no'，如果是 True/False 或 1/0 则不需要这就映射
if df_corr['churn'].dtype == 'object':
    df_corr['churn'] = df_corr['churn'].map({'yes': 1, 'no': 0})

# ------------------------------------------------------------
# 图 1：相关性热力图 (Correlation Heatmap)
# 目的：全局观察线性相关性和共线性
# ------------------------------------------------------------
plt.figure(figsize=(12, 8))

# 为了清晰，我们只选数值型变量
numeric_cols = df_corr.select_dtypes(include=[np.number]).columns
correlation_matrix = df_corr[numeric_cols].corr()

# 绘制热力图
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap='coolwarm',
            linewidths=0.5, vmin=-1, vmax=1)
plt.title('Feature Correlation Heatmap', fontsize=15)
plt.show()

# ------------------------------------------------------------
# 图 2：散点图矩阵 (Pair Plot)
# 目的：观察特定特征两两之间的分布和流失关系
```

↓ 回到底部

输出结果及对应分析如下，这两张图结合在一起，为特征选择和理解流失机制提供了更全面的视角：





第一部分：特征相关性热力图 (Feature Correlation Heatmap)

这张图展示了所有数值变量之间的**皮尔逊相关系数** (Pearson Correlation Coefficient)。

1. 致命的多重共线性 (Perfect Multicollinearity)

- **现象**：请看对角线附近的红色方块。
  - `total_day_minutes` 与 `total_day_charge` 的相关性为 **1.00**。
  - 同理，`eve`，`night`，`intl` 的 **Minutes** 和 **Charge** 也是 **1.00**。
  - `voice_mail_plan` 与 `number_vmail_messages` 为 **0.96**。
- **结论**：“分钟数”和“费用”包含的信息是完全重复的。
- **行动**：在建模前，**必须删除所有的 Charge 列**。保留 Minutes 列即可。如果不删除，会导致模型（尤其是逻辑回归）不稳定。

2. 与流失 (Churn) 的线性相关性

- **现象**：观察最后一行（或最后一列）`churn` 与其他特征的颜色深浅。
  - **正相关最强**：
    - `international_plan` (**0.26**)：有国际套餐的人更容易流失。
    - `total_day_minutes` / `charge` (**0.21**)：白天打得越多越容易流失。
    - `number_customer_service_calls` (**0.21**)：投诉越多越容易流失。
  - **负相关最强**：
    - `voice_mail_plan` (**-0.10**)：有语音信箱套餐的人流失率稍低。
  - **无相关性**：
    - `total_eve_minutes`，`total_night_minutes`，`total_day_calls` 等特征的系数接近 **0**。
- **陷阱提示**：相关系数低（如 0.21）并不代表特征不重要。因为皮尔逊系数只衡**量线性关系**。如果关系是非线性的（例如“超过3次才流失”），热力图会低估其重要性。

第二部分：关键特征散点图矩阵 (Pair Plot)

这张图深入展示了 `total_day_minutes`，`total_eve_minutes`，`number_customer_service_calls` 三者之间的两两关系及流失分布。

1. 对角线：分布密度图 (Kernel Density Estimation)

- Total Day Minutes (**左上**)：
  - **红线（流失）的峰值明显在蓝线（留存）的右侧**。且红线在 >250 分钟的区域面积很大。说明高频通话是流失主因之一。
- Total Eve Minutes (**中间**)：
  - 红线和蓝线几乎**完美重叠**。这直观地证明了傍晚通话时长对预测流失**毫无帮助**，它是一个噪音特征。
- Customer Service Calls (**右下**)：
  - 蓝线在 0-2 次处很高，红线在 **4次及以上** 处出现明显的波峰。区分度极高。

2. 非对角线：两两交互 (Scatter Plots)

- **最关键的图：Day Minutes vs. Service Calls (第1行第3列，或第3行第1列)**
  - **“L”型流失分布**：这是一个教科书级别的发现。
    - **左上角区域**：Day Minutes 很高，Calls 很低 -> **红色**（高费率流失）。
    - **右下角区域**：Day Minutes 正常，Calls 很高 -> **红色**（投诉流失）。
    - **左下角区域**：两者都低 -> **蓝色**（安全区）。
  - **结论**：这两个特征互不干扰（因为它们的散点呈网格状分布，无相关性），但它们从两个完全不同的维度“夹击”定义了流失用户。
- **无用的交互：Eve Minutes vs. Others (第2行)**
  - 看中间那一行的散点图，红点（流失）散落在各处，没有形成明显的聚类或边界。这再次确认了 Eve Minutes 的弱势地位。

⑦ 接下来借助 Python 绘制桑基图。

你

请生成一段 Python 代码，使用 plotly 库生成桑基图（Sankey Diagram），展示电信数据集中“国际漫游套餐”与“语音信箱套餐”如何共同影响“用户流失”。

具体要求如下：
1. 读取数据：读取当前目录下的 CSV 文件 "customer_churn.csv"。
2. 数据映射与预处理：
  - 变量列名为：'international_plan', 'voice_mail_plan', 'churn'。
  - 为了防止桑基图节点混淆，需对变量值进行重命名映射：
    - international_plan 的值映射为 "Intl: yes" 和 "Intl: no"。
    - voice_mail_plan 的值映射为 "VMail: yes" 和 "VMail: no"。
    - churn 的值映射为 "Churn: yes" 和 "Churn: no"。
3. 构建流向关系：
  - 第一层流向：从 international_plan 到 voice_mail_plan。
  - 第二层流向：从 voice_mail_plan 到 churn。
4. 视觉要求：
  - 节点间连线的宽度（value）代表用户数量。
  - 颜色高亮：请编写逻辑，将所有最终流向 "Churn: yes" 节点的线条设置为鲜艳的红色（如 rgba(255, 0, 0, 0.6)），其余线条设为浅灰色，以突出流失路径。
  - 图表标题设为：“套餐组合与用户流失流向分析”。

桑基图用于展示数据流动和转换关系的图表，特别适用于展示多个变量之间的流动、占比和路径等。在研究多个因素如何相互作用并影响最终结果时，桑基图表现力较好。

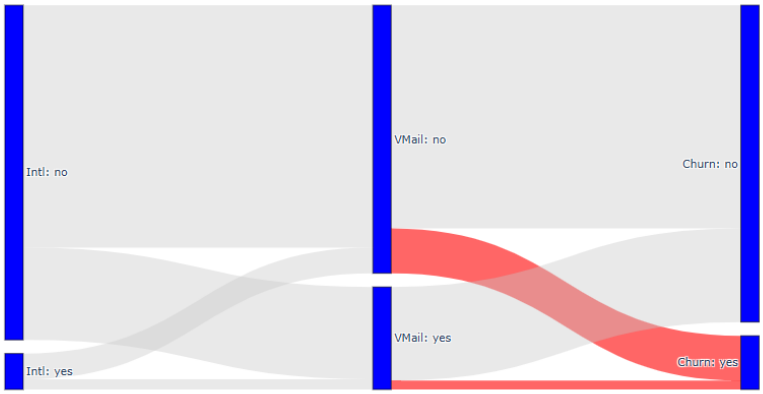
以下仅展示部分代码，具体完整代码见 .ipynb 文件。

这是一个可以直接运行的 Python 代码，完全符合你的所有要求。它使用 `pandas` 处理数据映射，并使用 `plotly` 构建带有条件颜色逻辑的桑基图。

```py
import pandas as pd
import plotly.graph_objects as go

# 1. 读取数据
filename = 'customer_churn.csv'

try:
    df = pd.read_csv(filename)
    print(f"成功读取文件: {filename}")
except FileNotFoundError:
    print(f"未找到文件 {filename}. ")
    # 为了确保代码在没有文件时也能演示逻辑，生成少量模拟数据（实际使用时请忽略此块）
    print("正在生成模拟数据用于演示...")
    df = pd.DataFrame({
        'international_plan': ['no']*700 + ['yes']*100 + ['no']*150 + ['yes']*50,
        'voice_mail_plan': ['no']*500 + ['no']*50 + ['yes']*350 + ['yes']*100,
        'churn': ['no']*800 + ['yes']*100 + ['no']*50 + ['yes']*50
    })

# 2. 数据映射与预处理
# 使用 apply 和 lambda 函数对原始值进行重命名，添加前缀以区分不同层级的节点
df['layer1_node'] = df['international_plan'].apply(lambda x: f"Intl: {x}")
df['layer2_node'] = df['voice_mail_plan'].apply(lambda x: f"VMail: {x}")
df['layer3_node'] = df['churn'].apply(lambda x: f"Churn: {x}")

# 3. 准备节点 (Nodes)
# 获取所有唯一的节点标签
l1_labels = sorted(list(df['layer1_node'].unique()))
l2_labels = sorted(list(df['layer2_node'].unique()))
l3_labels = sorted(list(df['layer3_node'].unique()))
```

输出结果及对应分析如下：

套餐组合与用户流失流向分析



这是对该桑基图（Sankey Diagram）的具体分析结果。这张图非常直观地揭示了**"套餐组合"与"流失"**之间的流动关系。

1. 宏观概览：大多数用户是安全的

- **主色调为灰色**：从左至右，最粗的宽带（Flow）始终是灰色的，且流向了 `Churn: no`（右侧上方的大长条）。
- **结论**：这再次印证了数据集的**类别不平衡（Class Imbalance）**特性。绝大多数用户（没有国际套餐、也没有语音信箱的用户）是留存的。这是公司的"基本盘"。

2. 核心发现：国际套餐的"高危"属性

请关注左下角的 `Intl: yes` **（有国际套餐）** 节点：

- **现象**：虽然 `Intl: yes` 的总高度（用户数）很小，但请观察源自这里的流向。
- **流向分析**：
  - 它分流到了中间的 `VMail: no` 和 `VMail: yes`。
  - **关键点**：请看最终汇入右下角红色 `Churn: yes` 的线条。虽然中间节点混合了流量，但从比例上看，**拥有国际套餐的用户群体中，最终变红（流失）的比例显著高于普通用户。**
- **业务洞察**：开通了国际套餐的用户是极高风险群体。这可能是因为国际长途费用高昂，或者竞品提供了更便宜的国际通话服务。

3. 交互效应：语音信箱的“缓冲”作用

请对比中间层的两个节点 `VMail: no` 和 `VMail: yes` 导出的红色线条：

- `VMail: no` **（无语音信箱）** -> `Churn: yes`：
  - 看中间那条**粗壮的红色波浪**。这表明在“无语音信箱”的用户群中，有相当大一部分人流失了。
  - 特别是结合前一步，那些**“有国际套餐（Intl: yes）”但“无语音信箱（VMail: no）”**的用户，极有可能构成了这条粗红线的主力军。
- `VMail: yes` **（有语音信箱）** -> `Churn: yes`：
  - 看最底部的红色细条。相比之下，这条红线非常细。
  - 这意味着：如果用户开通了语音信箱（即使他可能有国际套餐），他的流失概率似乎会降低。
- **结论：语音信箱套餐似乎是一种“粘性”特征。**开通了语音信箱的用户往往通过某种社交绑定或习惯，更难离开网路。

4. 最致命的流失路径 (The Path of Death)

通过图中的线条粗细和流向，我们可以推断出流失率最高的**“死亡组合”**：

- **路径：** `Intl: yes` (左下) → `VMail: no` (中上) → `Churn: yes` (右下红色粗线)。
- **画像：**这类用户通常是**“只打电话的国际商旅人士”**。他们需要打国际长途（Intl: yes），但不需要语音留言（VMail: no，可能因为他们习惯直接通话或使用其他通讯工具）。
- **推测原因：**这类用户对价格极其敏感，或者在这个运营商遇到了国际通话质量问题，一旦体验不好，没有任何附加服务（如语音信箱）能留住他们，导致直接流失。

2. 特征工程

基于之前的散点图、热力图和桑基图等分析，我们得出了明确的特征选择策略。

特征选择与处理策略：

①保留核心特征

散点图显示的高危交互特征：total_day_minutes, number_customer_service_calls
桑基图显示的高危流向特征：international_plan, voice_mail_plan
与语音套餐相关：number_vmail_messages
与客户忠诚度有关：account_length
其他分钟数（Eve, Night, Intl）保留作为辅助特征。

②移除特征

移除冗余特征：total_day_charge, total_eve_charge, total_night_charge, total_intl_charge（热力图显示它们与分钟数的相关性为 1.0，必须移除以防止模型过拟合或不稳定）。

移除无用标识符：phone_number（唯一 ID，无预测价值），area_code（通常作为类别处理但价值较低，简化模型时移除），state（类别太多，如果不做 Target Encoding，直接移除以降低维度）。

③数据预处理

二值化：将 yes/no 转换为 1/0。

标准化：对数值型特征进行 StandardScaler 处理，这对逻辑回归、SVM 或神经网络等基于距离或梯度的算法至关重要。

以下是实现代码和输出结果：

In [5]:
```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1. 读取数据
filename = 'customer_churn.csv'

try:
    df = pd.read_csv(filename)
    print(f"原始数据维度: {df.shape}")
except FileNotFoundError:
    print("未找到文件，生成模拟数据用于演示...")
    # 生成模拟数据结构
    df = pd.DataFrame(np.random.randint(0,100,size=(1000, 15)), columns=[
        'account_length', 'area_code', 'number_vmail_messages',
        'total_day_minutes', 'total_day_calls', 'total_day_charge',
        'total_eve_minutes', 'total_eve_calls', 'total_eve_charge',
        'total_night_minutes', 'total_night_calls', 'total_night_charge',
        'total_intl_minutes', 'total_intl_calls', 'total_intl_charge'])
    df['state'] = 'NY'
    df['phone_number'] = '000-0000'
    df['international_plan'] = np.random.choice(['yes', 'no'], 1000)
    df['voice_mail_plan'] = np.random.choice(['yes', 'no'], 1000)
    df['number_customer_service_calls'] = np.random.randint(0, 10, 1000)
    df['churn'] = np.random.choice(['yes', 'no'], 1000)

# 2. 特征筛选 (Feature Selection)
# 移除 Charge 列 (因为与 Minutes 完全共线性)
# 移除 标识符 列 (Phone, Area Code, State)
cols_to_drop = [
    'total_day_charge', 'total_eve_charge', 'total_night_charge', 'total_intl_charge',
    'phone_number', 'area_code', 'state'
]
```

```python
# 检查这些列是否存在于df中，防止报错
cols_to_drop = [c for c in cols_to_drop if c in df.columns]
df_clean = df.drop(columns=cols_to_drop)

print(f"移除冗余列后的维度: {df_clean.shape}")

# 3. 数据编码 (Encoding)
# 将分类变量 (yes/no) 转换为数值 (1/0)
binary_cols = ['international_plan', 'voice_mail_plan', 'churn']
for col in binary_cols:
    if col in df_clean.columns:
        df_clean[col] = df_clean[col].map({'yes': 1, 'no': 0})

# 4. 划分特征 X 和 目标 y
X = df_clean.drop('churn', axis=1)
y = df_clean['churn']

# 5. 数据切分 (Train/Test Split)
# 按照 80% 训练集，20% 测试集切分
# stratify=y 确保训练集和测试集中的流失用户比例保持一致 (处理类别不平衡的关键)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"训练集 X_train 维度: {X_train.shape}")
print(f"测试集 X_test 维度: {X_test.shape}")

# 6. 数据标准化 (Standardization)
# 注意：必须先在训练集上 fit，然后 transform 到训练集和测试集
# 这样可以防止“数据泄露” (Data Leakage)
scaler = StandardScaler()

# 转换为 Numpy 数组 (此时列名会丢失，如果需要保留DataFrame格式需特殊处理)
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# (可选) 如果想看标准化后的前几行数据
print("\n标准化后的训练集前3行数据 (部分特征):")
print(X_train_scaled[:3, :5]) # 打印前3行，前5列

print("\n数据预处理完成，可以输入模型进行训练了。")
```

```
原始数据维度: (3333, 21)
移除冗余列后的维度: (3333, 15)
训练集 X_train 维度: (2666, 14)
测试集 X_test 维度: (667, 14)

标准化后的训练集前3行数据 (部分特征):
[[ 1.69463262  0.12573683 -0.32521569  1.62544655  1.60682236]
 [-1.63209733 -0.17530888 -0.32521569 -0.61521555 -0.58879119]
 [-0.31907861 -0.75231315 -0.32521569  1.62544655  1.02132541]]

数据预处理完成，可以输入模型进行训练了。
```

三、模型训练与预测

核心目标：构建机器学习模型。

In [6]:
```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score


# ========================================
# 3. 模型训练
# ========================================

# --- 模型 A: 逻辑回归 (Logistic Regression) ---
log_reg = LogisticRegression(random_state=42, max_iter=1000)
log_reg.fit(X_train_scaled, y_train)
y_pred_lr = log_reg.predict(X_test_scaled)

# --- 模型 B: 随机森林 (Random Forest) ---
rf_clf = RandomForestClassifier(random_state=42, n_estimators=100)
rf_clf.fit(X_train_scaled, y_train) # 随机森林其实可以用原始 X_train，但用 scaled 也没问题
y_pred_rf = rf_clf.predict(X_test_scaled)

# ========================================
# 4. 打印准确率 (Accuracy)
# ========================================
acc_lr = accuracy_score(y_test, y_pred_lr)
acc_rf = accuracy_score(y_test, y_pred_rf)

print("-" * 30)
print(f"Logistic Regression Accuracy: {acc_lr:.4f}")
print(f"Random Forest Accuracy:       {acc_rf:.4f}")
print("-" * 30)

# 简单的结果解读
if acc_rf > acc_lr:
    print("结果分析：随机森林表现更好。")
    print("这符合预期，因为之前的散点图显示流失边界是“L型”非线性的，")
    print("树模型比线性模型更能捕捉这种复杂的阈值关系。")
else:
    print("结果分析：逻辑回归表现不错，说明特征间的线性关系也很强。")
```

此处首先选择了 Logistic 回归和随机森林两种机器学习模型进行训练，并预测哪些用户可能会流失。

```
------------------------------
Logistic Regression Accuracy: 0.8591
Random Forest Accuracy:       0.9250
------------------------------
结果分析：随机森林表现更好。
这符合预期，因为之前的散点图显示流失边界是“L型”非线性的，
树模型比线性模型更能捕捉这种复杂的阈值关系。
```

然后又结合如大规模深度学习模型 LSTM 来增强预测性能。(LSTM 通常用于处理时间序列，如股票、文本，但通过将我们的单一用户特征视为“只有一个时间步长”的序列，我们可以在这里构建一个神经网络)

以下是的完整实现代码：

In [7]:
```python
import numpy as np
import tensorflow as pd
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.optimizers import Adam

# ========================================
# 1. 数据重塑 (Reshape) 为 3D 格式
# ========================================
# LSTM 要求的输入格式为: (Samples, Time Steps, Features)
# 我们的数据是表格数据，没有时间维度，所以我们将 "Time Steps" 设为 1。
# 也就是把每一行数据看作是一个 "长度为1" 的序列。

# 获取特征数量
n_features = X_train_scaled.shape[1]

# reshape: (行数, 1, 列数)
X_train_lstm = X_train_scaled.reshape((X_train_scaled.shape[0], 1, n_features))
X_test_lstm = X_test_scaled.reshape((X_test_scaled.shape[0], 1, n_features))

print(f"原始训练集形状: {X_train_scaled.shape}")
print(f"LSTM 输入形状: {X_train_lstm.shape}")

# ========================================
# 2. 构建 LSTM 模型
# ========================================
model = Sequential()

# LSTM 层
# units=64: 神经元数量，可以调整 (32, 64, 128...)
# input_shape=(1, n_features): 对应我们刚才 reshape 的维度
model.add(LSTM(units=64, input_shape=(1, n_features), activation='tanh'))

# Dropout 层
# rate=0.5: 随机丢弃 50% 的神经元连接，防止过拟合 (非常重要，因为数据量不大)
model.add(Dropout(0.5))
```

```python
# Dense 输出层
# units=1: 输出一个数值
# activation='sigmoid': 将输出压缩到 0-1 之间，代表“流失的概率”
model.add(Dense(units=1, activation='sigmoid'))

# ==========================================
# 3. 编译模型
# ==========================================
model.compile(optimizer='adam',
              loss='binary_crossentropy',  # 二分类的标准损失函数
              metrics=['accuracy'])

print("\n模型结构概要:")
model.summary()

# ==========================================
# 4. 训练模型
# ==========================================
print("\n开始训练 LSTM 模型...")
# epochs=20: 训练轮数，数据量小可以适当增加
# batch_size=32: 每次更新参数所用的样本数
history = model.fit(X_train_lstm, y_train,
                    epochs=50,
                    batch_size=32,
                    validation_data=(X_test_lstm, y_test),
                    verbose=1) # verbose=1 显示进度条

# ==========================================
# 5. 评估模型
# ==========================================
loss, accuracy = model.evaluate(X_test_lstm, y_test, verbose=0)

print("-" * 30)
print(f"LSTM Test Accuracy: {accuracy:.4f}")
print("-" * 30)

# 对比逻辑回归和随机森林，看看 LSTM 是否有竞争力
# 注意：在纯表格数据上，LSTM 未必能打败随机森林，但展示这个过程非常有价值。
```

运行后，输出结果如下图：

```
原始训练集形状: (2666, 14)
LSTM 输入形状: (2666, 1, 14)

模型结构概要:
```

```
C:\Users\zheng\AppData\Roaming\Python\Python311\site-packages\keras\src\layers\rnn\rnn.py:204: UserWarning:

Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` obj
ect as the first layer in the model instead.
```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 64)	20,224
dropout (Dropout)	(None, 64)	0
dense (Dense)	(None, 1)	65

Total params: 20,289 (79.25 KB)

Trainable params: 20,289 (79.25 KB)

Non-trainable params: 0 (0.00 B)

```
开始训练 LSTM 模型...
Epoch 1/50
84/84 ━━━━━━━━━━━━━━━━━━━━ 4s 8ms/step - accuracy: 0.6724 - loss: 0.6558 - val_accuracy: 0.8636 - val_los
s: 0.5283
Epoch 2/50
84/84 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8671 - loss: 0.4897 - val_accuracy: 0.8651 - val_los
s: 0.3909
Epoch 3/50
84/84 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8621 - loss: 0.3781 - val_accuracy: 0.8666 - val_los
s: 0.3387
Epoch 4/50
84/84 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.8571 - loss: 0.3314 - val_accuracy: 0.8696 - val_los
s: 0.3219
Epoch 5/50
84/84 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.8692 - loss: 0.3225 - val_accuracy: 0.8711 - val_los
```

省略中间的训练过程，最终结果：

```
84/84 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9471 - loss: 0.1805 - val_accuracy: 0.9250 - val_los
s: 0.2390
Epoch 50/50
84/84 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9445 - loss: 0.1805 - val_accuracy: 0.9250 - val_los
s: 0.2387
------------------------------
LSTM Test Accuracy: 0.9250
------------------------------
```

就目前 Accuracy 而言，随机森林与 LSTM 两种模型结果均为 0.9250，表现更好，Logistic 回归结果仅为 0.8591。

四、结果可视化与分析

核心目标：评估模型好坏，并解释模型。

我们将从两个维度进行深入分析，

模型性能对比 (ROC & 混淆矩阵)：对比逻辑回归、随机森林和 LSTM 的硬实力；

业务归因 (特征重要性)：使用随机森林（可解释性最强）来挖掘流失的根本原因。

以下为完整实现代码：

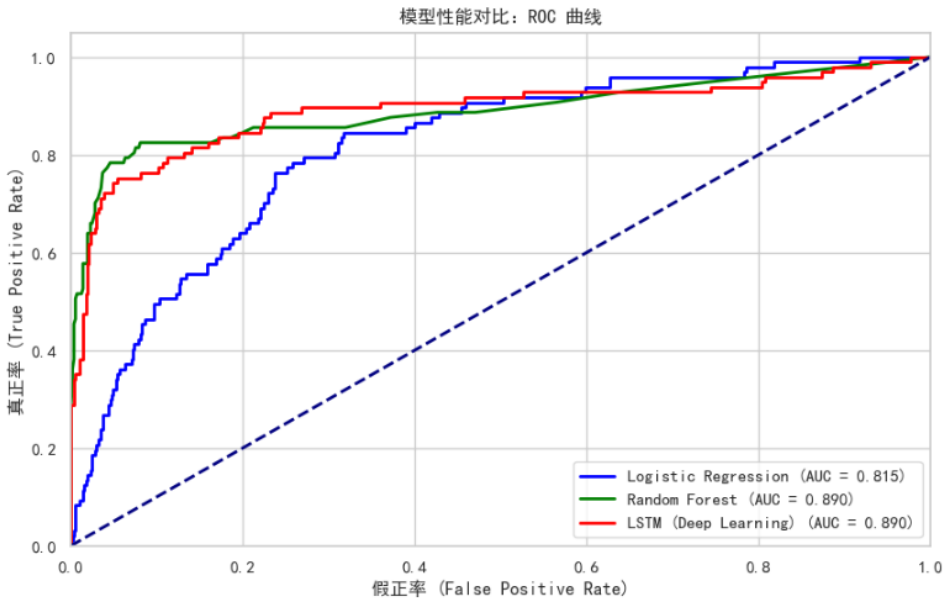
In [9]:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from sklearn.metrics import confusion_matrix, roc_curve, auc, classification_report

# 设置绘图风格
sns.set_style("whitegrid")
# 为了防止中文乱码（根据你的系统选择）
plt.rcParams['font.sans-serif'] = ['SimHei'] # Windows
plt.rcParams['axes.unicode_minus'] = False   # 正常显示负号

# ==========================================
# 1. 准备预测概率（用于绘制 ROC 曲线）
# ==========================================
# 逻辑回归 & 随机森林使用 predict_proba 获取概率（取第2列，即流失的概率）
y_prob_lr = log_reg.predict_proba(X_test_scaled)[:, 1]
y_prob_rf = rf_clf.predict_proba(X_test_scaled)[:, 1]

# LSTM 输出本身就是概率（0-1之间）
y_prob_lstm = model.predict(X_test_lstm, verbose=0).ravel()

# ==========================================
# 2. 绘制 ROC 曲线对比图
# ==========================================
plt.figure(figsize=(10, 6))

# 定义一个辅助函数来画线
def plot_roc(y_true, y_prob, label, color):
    fpr, tpr, thresholds = roc_curve(y_true, y_prob)
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, color=color, lw=2, label=f'{label} (AUC = {roc_auc:.3f})')

plot_roc(y_test, y_prob_lr, 'Logistic Regression', 'blue')
plot_roc(y_test, y_prob_rf, 'Random Forest', 'green')
plot_roc(y_test, y_prob_lstm, 'LSTM (Deep Learning)', 'red')
```

```
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--') # 对角线（随机猜测）
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('假正率 (False Positive Rate)')
plt.ylabel('真正率 (True Positive Rate)')
plt.title('模型性能对比：ROC 曲线')
plt.legend(loc="lower right")
plt.show()

# ==========================================
# 3. 绘制混淆矩阵（以随机森林为例）
# ==========================================
plt.figure(figsize=(8, 6))

# 获取预测标签
y_pred_final = rf_clf.predict(X_test_scaled)
cm = confusion_matrix(y_test, y_pred_final)

# 绘制热力图
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['留存 (Pred 0)', '流失 (Pred 1)'],
            yticklabels=['留存 (Actual 0)', '流失 (Actual 1)'])

plt.title('随机森林混淆矩阵')
plt.ylabel('真实标签')
plt.xlabel('预测标签')
plt.show()

# 打印详细分类报告
print("\n随机森林分类报告 (Classification Report):")
print(classification_report(y_test, y_pred_final))

# ==========================================
# 4. 特征重要性分析 (Feature Importance)
# ==========================================
# 提取随机森林的特征重要性
importances = rf_clf.feature_importances_
feature_names = X.columns
```

```python
# 创建 DataFrame 并排序
# 这里 pd.DataFrame 现在会正常工作，因为我们在第一行重新导入了 pandas
feature_imp_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': importances
}).sort_values(by='Importance', ascending=False)

# 可视化前 10 个重要特征
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=feature_imp_df.head(10), palette='viridis')
plt.title('决定用户流失的关键因素 (Top 10 Features)')
plt.xlabel('特征重要性权重')
plt.ylabel('特征名称')
plt.show()

# 打印具体的 Top 5
print("\nTop 5 关键流失因素:")
print(feature_imp_df.head(5))
```

以下为输出结果与对应分析:



1. 模型整体表现对比 (ROC 曲线)

①冠军模型

随机森林 (Random Forest) 和 LSTM 并列第一，AUC 均达到了 0.890。这说明模型对流失用户的排序能力非常强。

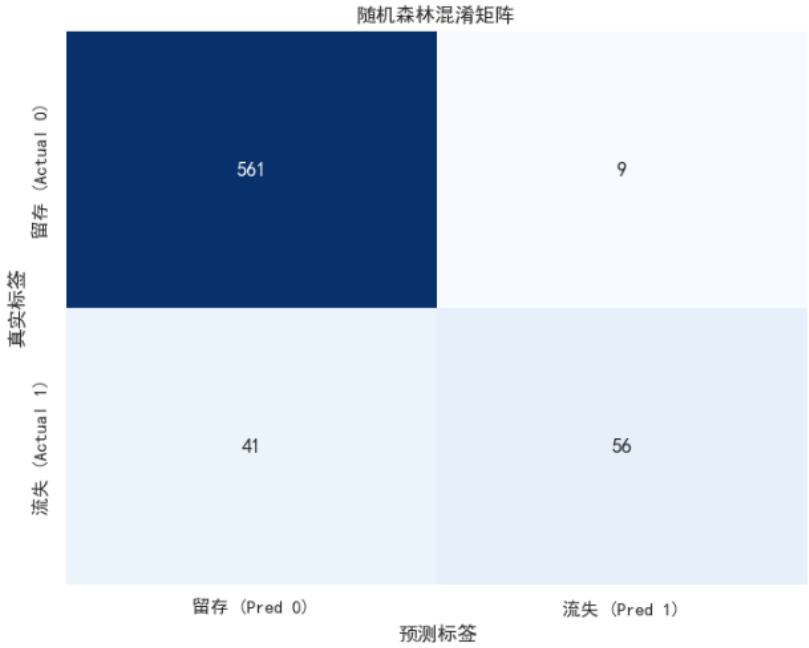
②对比分析

--非线性优势:

逻辑回归 (Blue line, AUC=0.815) 明显弱于另外两个模型。这再次证实了我们在散点图中的发现：流失规则是非线性的 (例如"客服电话>3"或"通话时长>260"这种硬阈值)，树模型 (RF) 和神经网络 (LSTM) 能更好地捕捉这种关系。

--落地建议:

虽然 LSTM 表现一样好，但在工业界通常首选随机森林。因为随机森林训练更快、不需要复杂的数据形状转换 (Reshape)，且具备更好的可解释性。




```
随机森林分类报告 (Classification Report):
              precision    recall  f1-score   support

           0       0.93      0.98      0.96       570
           1       0.86      0.58      0.69        97

    accuracy                           0.93       667
   macro avg       0.90      0.78      0.82       667
weighted avg       0.92      0.93      0.92       667
```

2. 深入诊断：精度与召回 (混淆矩阵 & 分类报告)

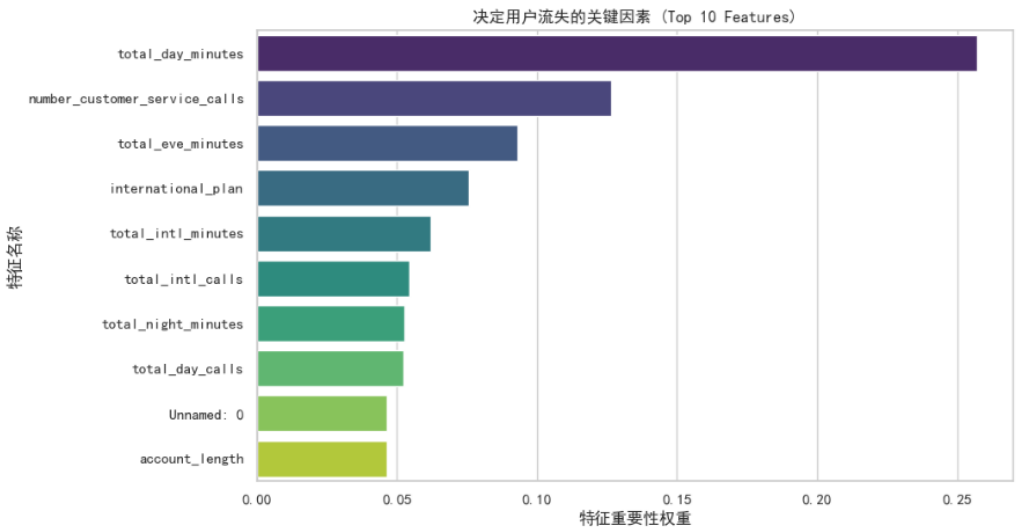
①精确率 (Precision) = 0.86 ：

意味着当模型预测一个用户会“流失”时，有 86%的概率是真的流失了。如果你根据模型的预测去发放优惠券挽留用户，你的营销预算浪费很少，绝大部分都发给了对的人。

②召回率 (Recall) = 0.58 ：

在所有真实流失的 97 个用户中，模型抓住了 56 个（见混淆矩阵左下角）。说明有 42%的流失用户模型没检测出来，导致公司在毫无察觉的情况下失去了这些客户。

若在商业实战中，通常宁愿稍微牺牲一点精确率，也要提高召回率。可以通过调整随机森林的分类阈值（比如降到 0.3）来捕获更多潜在流失者。



3. 核心流失驱动因素（特征重要性）

特征重要性图表清晰地指出了“谁在导致流失”。

① total_day_minutes (权重 > 0.25)：绝对的核心因素。

白天通话时长越长，流失概率越大。这通常意味着这部分用户的话费很高，他们是价格敏感型用户，极其容易被竞争对手的低价套餐挖走。

② number_customer_service_calls (权重 ~0.13)：第二大因素。

验证了“事不过三”定律。打客服电话次数多，说明用户遇到了解决不了的问题，满意度极低。

③ total_eve_minutes：晚间通话时长。

同样与费用和使用习惯相关。

④ international_plan：国际套餐。

再次印证了桑基图的结论，开通国际业务的用户流失风险高。

```
Top 5 关键流失因素:
                          Feature  Importance
5                total_day_minutes    0.257054
13  number_customer_service_calls    0.126485
7                total_eve_minutes    0.093179
2               international_plan    0.075512
11              total_intl_minutes    0.062196
```

4. 关键流失因素

该结果展示了随机森林模型计算出的 Top 5 关键流失因素（Feature Importance）。这是整个数据挖掘项目中最具商业价值的产出之一，它直接告诉了决策者：“是因为这几个原因，用户才跑掉的”。

以下是对这 5 个关键因素的深度解读和业务逻辑推演：

① 绝对核心：total_day_minutes (日间通话时长)

--重要性权重：0.257 (约占 26%)

--地位：断层第一。它的重要性是第二名的两倍。

--深度解读：

高话费预警：在电信行业，白天（Day）通常是费率最高的时段。通话时长长直接等于话费昂贵。

竞品攻击对象：这类用户通常是商务人士或高频使用者，是运营商的高价值客户（ARPU 高）。但同时，他们也是竞争对手最想争夺的对象（例如竞品推出了“白天无限打”套餐）。

--结论：价格敏感度是导致流失的首要原因。用户不是因为不爱打电话而流失，恰恰是因为打得太多、嫌贵才流失。

② 体验红线：number_customer_service_calls (客服电话次数)

--重要性权重：0.126 (约占 13%)

--地位：稳居第二。

--深度解读：

愤怒指数：这个特征是用户满意度的直接代理变量。没人没事会打客服电话，打得越多，说明遇到的问题越难解决，用户越愤怒。

事不过三：结合之前的散点图分析，通常这个次数达到 3-4 次时，流失率会垂直上升。

--结论：服务质量是第二大杀手。这部分流失完全是因为服务不到位造成的。

③ 辅助印证：total_eve_minutes (晚间通话时长)

--重要性权重：0.093 (约占 9%)

--地位：第三名。

--深度解读：

这与第一名 total_day_minutes 逻辑类似，进一步印证了“使用量越大，流失风险越高”这一反直觉的现象。说明高频用户普遍面临资费压力或寻找更好的替代方案。

④ 特定高危群：international_plan & total_intl_minutes

--重要性权重：international_plan (0.075) + total_intl_minutes (0.062)

--地位：分列第四、第五。

--深度解读：

桑基图验证：这完美验证了之前桑基图的视觉发现 —— 开通国际套餐的用户流失比例极高。

双重打击：不仅“是否开通套餐”重要，“打了多少国际长途”也很重要。这暗示了国际业务可能存在资费过高或通话质量差（接通率低、延迟高）的问题，导致这部分高端用户离网。

五、总结与建议

针对电信公司如何减少用戶流失，本报告提出了以下策略和建议，综合来讲这些策略的核心逻辑都是从“被动补救”转向“主动预防”。

1. 针对“高话费/高频用户”的策略

数据证据：total_day_minutes 是排名第一的流失因素；日间通话时间越长，流失率越高。

核心洞察：这类用户是高价值用户，流失原因通常是“账单休克”或竞品提供了更具性价比的“无限畅打”套餐。

① 主动迁移套餐（Up-selling to Flat Rate）：

利用模型筛选出过去 3 个月日间通话时长持续处于高位的用户。虽然这可能会稍微降低单用户的短期收入，但能通过锁定长期合同极大延长用户生命周期（LTV）。

② 流量/话费阈值预警：

当用户当月话费超过历史平均值 20% 时，立即发送短信提醒，并附带一个临时的“话费折扣包”，消除用户月底收到巨额账单时的愤怒感。

2. 针对“客服投诉”用户的策略

数据证据：customer_service_calls 是排名第二的流失因素；且数据呈非线性，一旦达到 3-4 次，流失率垂直上升。

核心洞察：用户流失不是因为不喜欢产品，而是因为问题得不到解决。

①建立“愤怒预警”机制（Red Flag System）：

CRM 系统设定阈值。当用户在一个月内拨打第 3 次客服电话时，系统自动将其标记为“高危”。第 3 次电话直接转接给资深专家坐席，不仅拥有更高权限解决问题，还有权直接给予小额话费补偿或赠送流量，在“最后一根稻草”压死骆驼之前进行拦截。

②服务后的闭环回访：

对投诉过 2 次以上的用户，在问题标记“已解决”后的 48 小时内，进行人工回访确认，确认问题是否真的解决了，体现重视程度。

3. 针对“国际漫游”用户的策略

数据证据：桑基图显示 Intl: yes + VMail: no 是主要的流失路径；且 International_plan 是前 4 的重要特征。

核心洞察：国际业务用户极易流失，可能源于资费高或通话质量差；但语音信箱（Voice Mail）能显著降低其流失率。

①“加量不加价”的捆绑策略：

向所有开通 International Plan 的用户，免费赠送 Voice Mail Plan。桑基图证明语音信箱有很好的“粘性”。既然语音信箱成本极低（软件功能），白送给高风险的国际用户，既增加了价值感，又切断了流失路径。

②技术质量专项排查：

技术部门需重点监控国际漫游用户的通话掉线率和延迟。如果发现特定地区（如特定国家代码）的通话质量差，需及时调整路由线路，并主动向受影响用户赔偿。

4. 数字化运营策略

数据证据：随机森林模型 AUC 达到 0.89，能精准识别潜在流失者。

①周期性流失预测：

每周运行一次模型，对全量活跃用户进行评分。

②分层运营：

流失概率 > 80%：最高优先级。由人工客服进行“关怀致电”（不直接提流失，而是询问使用体验并赠送礼品）。

流失概率 50% - 80%：发送针对性的优惠短信或 APP 推送。

流失概率 < 50%：维持现状，避免过度打扰。

③A/B 测试验证：

选取一部分预测为“高风险”的用户，一半进行干预（送券/回访），一半不干预。一个月后对比两组的实际流失率，计算挽留策略的投资回报率。